Framework for Externalised Graph-Based Reasoning for Document-Level Question Answering

Akshat Kotadia
IIIT Hyderabad
2025201005

Gaurav Patel
IIIT Hyderabad
2025201065

Om Mehra
IIIT Hyderabad
2025201008

Hardik Kothari
IIIT Hyderabad
2025201046

Project Repository:
https://github.com/Akshat-A-K/Graph-based-QA/tree/project_mid

Abstract

Question answering over long documents remains a challenging task due to the need for effective information retrieval and multi-hop reasoning across dispersed pieces of evidence. In this work, we present a graph-based document question answering system that combines traditional information retrieval techniques with semantic representation learning and graph-based reasoning. The proposed system first extracts textual content from documents and organises it into sentences, which are treated as nodes in a Document Reasoning Graph (DRG). Semantic relationships between sentences are established using sentence embeddings, forming edges based on similarity thresholds. From these sentences, important spans such as entities and key phrases are extracted to construct a Span Graph, enabling more fine-grained reasoning over potential answer candidates. To identify relevant information for a given query, the system employs a hybrid retrieval strategy that integrates BM25 lexical matching with dense semantic embeddings. Graph centrality measures and traversal strategies are then used to identify important evidence nodes and support multi-hop reasoning across the document. Finally, an extractive transformer-based question answering model selects the most plausible answer span from the retrieved evidence. Experimental analysis demonstrates that combining graph structure with hybrid retrieval improves the system's ability to locate relevant evidence and produce accurate answers from complex documents. The proposed framework provides an interpretable and extensible approach for document-level question answering by leveraging both semantic similarity and structural reasoning within a unified graph representation.

1 Introduction

1.1 Project Idea

Question Answering (QA) systems aim to automatically extract accurate answers from large collections of textual information. While modern transformer-based models have significantly improved extractive question answering (Devlin et al., 2019), they often struggle when dealing with long documents where relevant information is scattered across multiple sentences or sections. In such cases, simple retrieval methods may fail to capture relationships between different pieces of evidence required to answer complex queries.

This project proposes a **Graph-Based Document Question Answering System** that integrates document structure, semantic similarity, and graph reasoning to improve answer retrieval from long documents. The system first extracts textual content from documents and divides it into sentences. These sentences are represented as nodes in a *Document Reasoning Graph (DRG)*, where edges are formed based on semantic similarity between sentences using sentence embeddings. In addition, important spans such as named entities and key phrases are extracted to construct a *Span Graph*, enabling more fine-grained reasoning over potential answer candidates.

To retrieve relevant information for a given question, the system combines traditional lexical retrieval using BM25 with dense semantic embeddings. Graph-based reasoning techniques such as centrality analysis and graph traversal are then applied to identify important evidence nodes. Finally, a transformer-based extractive question answering model selects the most probable answer span from the retrieved evidence. This hybrid framework allows the system to leverage both semantic understanding and structural relationships within the document.

1.2 Motivation

The rapid growth of digital information has created a need for intelligent systems that can efficiently retrieve precise answers from large volumes of text. Traditional search engines return documents or passages, leaving the user to manually locate the required information. Question answering systems attempt to automate this process by directly extracting answers from the text.

However, many real-world questions require reasoning across multiple pieces of information that may appear in different parts of a document. Standard retrieval-based QA systems often treat sentences independently and fail to capture relationships between them. This limitation becomes more evident when dealing with long documents, technical papers, or structured reports where relevant information may be distributed across sections.

Graph-based representations provide a natural way to model relationships between textual elements such as sentences, entities, and phrases. By representing the document as a graph and analyzing the connections between nodes, it becomes possible to perform multi-hop reasoning and identify important evidence for answering complex queries. Motivated by this idea, the proposed system integrates graph structures with modern NLP techniques to improve document-level question answering.

1.3 Objectives

The main objectives of this project are as follows:

- To design a document processing pipeline that extracts sentences and meaningful spans from textual documents.
- To construct graph-based representations of documents using semantic similarity between sentences and spans.
- To develop a hybrid retrieval mechanism combining BM25 lexical retrieval and dense semantic embeddings.
- To apply graph-based reasoning techniques to identify important evidence nodes for answering queries.
- To implement an extractive transformer-based model that selects the most relevant answer span from the retrieved evidence.
- To build a complete end-to-end system capable of answering user queries from long documents efficiently.

2 Background and Related Work

2.1 Background

2.1.1 Question Answering Systems

Question Answering (QA) systems retrieve or generate answers to natural language questions from text or structured sources. Modern pipelines typically combine a retrieval stage that selects candidate passages with a reader that extracts or generates answers; extractive readers are common in document-based QA to keep responses grounded.

2.1.2 Multi-hop Question Answering

Multi-hop QA requires linking information across multiple sentences or documents to reach an answer. Systems must identify relevant evidence and compose reasoning steps that connect disparate facts.

2.1.3 Information Retrieval in QA

Retrieval narrows large corpora to a manageable set of candidates. Lexical methods such as BM25 remain strong baselines (Robertson and Zaragoza, 2009); dense retrieval (learned vector encoders) improves semantic matching beyond keyword overlap.

2.1.4 Graph-Based Text Representation

Graph representations model textual units (sentences, entities, spans) as nodes with edges encoding relations, enabling traversal and path-based reasoning—useful for locating and combining multi-hop evidence.

2.1.5 Transformer-Based Question Answering Models

Transformer models (e.g., BERT and variants) are commonly fine-tuned as extractive readers to predict answer spans from question-context pairs. Combining effective retrieval (lexical or dense) with transformer readers is the standard architecture for document-level QA.

2.2 Related Work

Research in question answering has evolved significantly with the development of multi-hop reasoning datasets, graph-based reasoning architectures, and retrieval-based question answering systems. This section reviews previous work closely related to our approach.

2.2.1 Multi-hop Question Answering

Multi-hop question answering requires models to integrate information from multiple pieces of evidence to derive the final answer. One of the most influential datasets in this domain is HotpotQA (Yang et al., 2018), which was specifically designed to evaluate multi-hop reasoning capabilities of QA systems. Unlike earlier datasets that primarily focus on single-paragraph comprehension, HotpotQA provides sentence-level supporting facts and requires models to perform reasoning across multiple documents. The dataset contains over 113,000 Wikipedia-based question–answer pairs and includes both bridge and comparison questions, making it suitable for evaluating explainable reasoning.

Several recent works have further emphasized the importance of explicitly modeling reasoning structures. For instance, the GRS-QA (Pahilajani et al., 2024) dataset introduces reasoning graphs that represent the logical paths connecting supporting evidence required to answer a question. These graph-based annotations allow researchers to analyse reasoning patterns and evaluate the ability of models to follow structured reasoning processes.

2.2.2 Graph-based Reasoning for Question Answering

Graph representations have been widely explored for modelling relationships between textual elements in question answering systems. CogQA (Ding et al., 2019) proposes a cognitive graph framework in which entities and passages are represented as nodes while edges capture relationships between them. The system performs multi-hop reasoning by iteratively expanding the graph to discover relevant evidence.

Similarly, (Tu et al., 2019) introduced a heterogeneous graph network for multi-hop question answering, where questions, sentences, and entities are represented as different node types within a unified graph structure. Graph neural networks are then applied to propagate information across nodes, enabling reasoning across multiple evidence sources.

Graph-based modeling has also been applied to text representation tasks beyond QA. (Luan et al., 2019) proposed a dynamic span graph framework in which textual spans are treated as nodes and relations between spans form edges. The graph structure allows contextual information to propagate between related spans, improving performance in information extraction tasks.

(Fang et al., 2020) proposed a Hierarchical Graph Network (HGN) specifically designed for HotpotQA, which constructs a multi-level graph comprising question, paragraph, sentence, and entity nodes, and propagates information across levels using graph neural network layers to perform multi-hop reasoning. In contrast to this learned approach, the system presented in this work applies heuristic graph traversal with semantic re-scoring, making the reasoning process more transparent and directly applicable without task-specific training on HotpotQA.

2.2.3 Retrieval-based Question Answering

Open-domain QA systems typically rely on retrieval mechanisms to identify relevant documents before extracting answers. Traditional information retrieval methods such as BM25 remain widely used due to their effectiveness in ranking documents based on lexical similarity.

Recent research has introduced dense retrieval models that encode questions and passages into vector representations. Dense Passage Retrieval (DPR) (Karpukhin et al., 2020) demonstrated that neural retrieval models significantly improve open-domain question answering performance by capturing semantic similarity beyond keyword matching.

Retrieval-augmented language models such as REALM (Guu et al., 2020) and RAG (Lewis et al., 2021) have further advanced open-domain QA.

Transformer-based language models have also significantly advanced reading comprehension in question answering systems. BERT introduced a bidirectional transformer architecture that can be fine-tuned for extractive QA tasks by predicting the start and end positions of answer spans within a context. BERT-based models have achieved state-of-the-art performance on several benchmark datasets and have become a standard component in many QA pipelines.

Our work builds upon these developments by combining lexical retrieval with semantic similarity and graph-based reasoning. Unlike many existing approaches that rely on heavy training of neural models, our system focuses on constructing reasoning graphs from document sentences and spans, enabling structured reasoning over retrieved evidence when answering questions from long documents.

3 Dataset

We evaluate the proposed graph-based document QA system using the HotpotQA dataset, a benchmark designed for multi-hop question answering where each question often requires reasoning across multiple supporting passages. Evaluation is executed with the driver script `hotpot_dataset.py`, which converts HotpotQA examples into the repository's document/graph format and runs the full DRG → SpanGraph → EnhancedHybridReasoner → select_answer pipeline.

3.1 Dataset Source

HotpotQA is a publicly released dataset of crowd-sourced questions that require multi-hop reasoning across Wikipedia paragraphs. We use the official train and dev splits for development and evaluation and follow the dataset's recommended evaluation protocol.

3.2 Dataset Statistics

Key corpus statistics:

- **Train-easy:** 18,089 examples.
- **Train-medium:** 56,814 examples.
- **Train-hard:** 15,661 examples.
- **Dev:** 7,405 examples.
- **Test:** 7,405 examples each.
- **Total (all splits):** 112,779 examples.

3.3 Preprocessing

For each HotpotQA example we run the repository pipeline to convert dataset instances into the document-graph format expected by our system:

- Normalise and clean text (Unicode normalisation, whitespace trimming).
- Segment contexts into sections and sentences.
- Build sentence-level nodes and the Document Reasoning Graph and compute sentence embeddings with the configured sentence-transformer.
- Extract candidate spans (entities, noun phrases, numerics) and construct the Span Graph.
- Index sentences/spans for BM25 (sparse) retrieval and precompute dense sentence/span embeddings for semantic retrieval.
- Persist indices and embeddings to disk to speed repeated evaluation runs.

We evaluate using a stratified subset of HotpotQA consisting of 1,000 examples sampled from each difficulty level (easy, medium, hard), for a total of 3,000 examples. Examples were selected by random stratified sampling with seed 42 to ensure reproducibility. This reduced subset was chosen to limit computational cost and enable repeated runs; all reported results use this subset unless explicitly stated otherwise.

3.4 Evaluation Protocol

We evaluate the system end-to-end and report the following:

- **Answer metrics:** Exact Match (EM) and token-level F1 for extractive answers.
- **Retrieval diagnostics:** Recall@k for BM25, dense, and hybrid retrieval before answer extraction.
- **Supporting facts:** Supporting-fact retrieval accuracy and analysis of graph traversal/support paths used by the reasoner.
- **Operational metrics:** Average reasoning depth, average time per question, and confidence distributions for predicted answers.
- **Ablations and per-level reporting:** Compare BM25-only, dense-only, hybrid, and graph-guided variants; report EM/F1 per difficulty level (easy/medium/hard).

All experimental metrics (EM, F1, Recall@k, and per-level reporting) are computed on the stratified 3,000-example subset (1k per level). Per-level metrics are reported over the corresponding 1,000-example subsets.

4 System Overview

4.1 Pipeline Overview

Figure 1 illustrates the end-to-end pipeline of the proposed Graph-Based Document Question Answering (GBDQA) system. The system processes input documents and queries through a series of

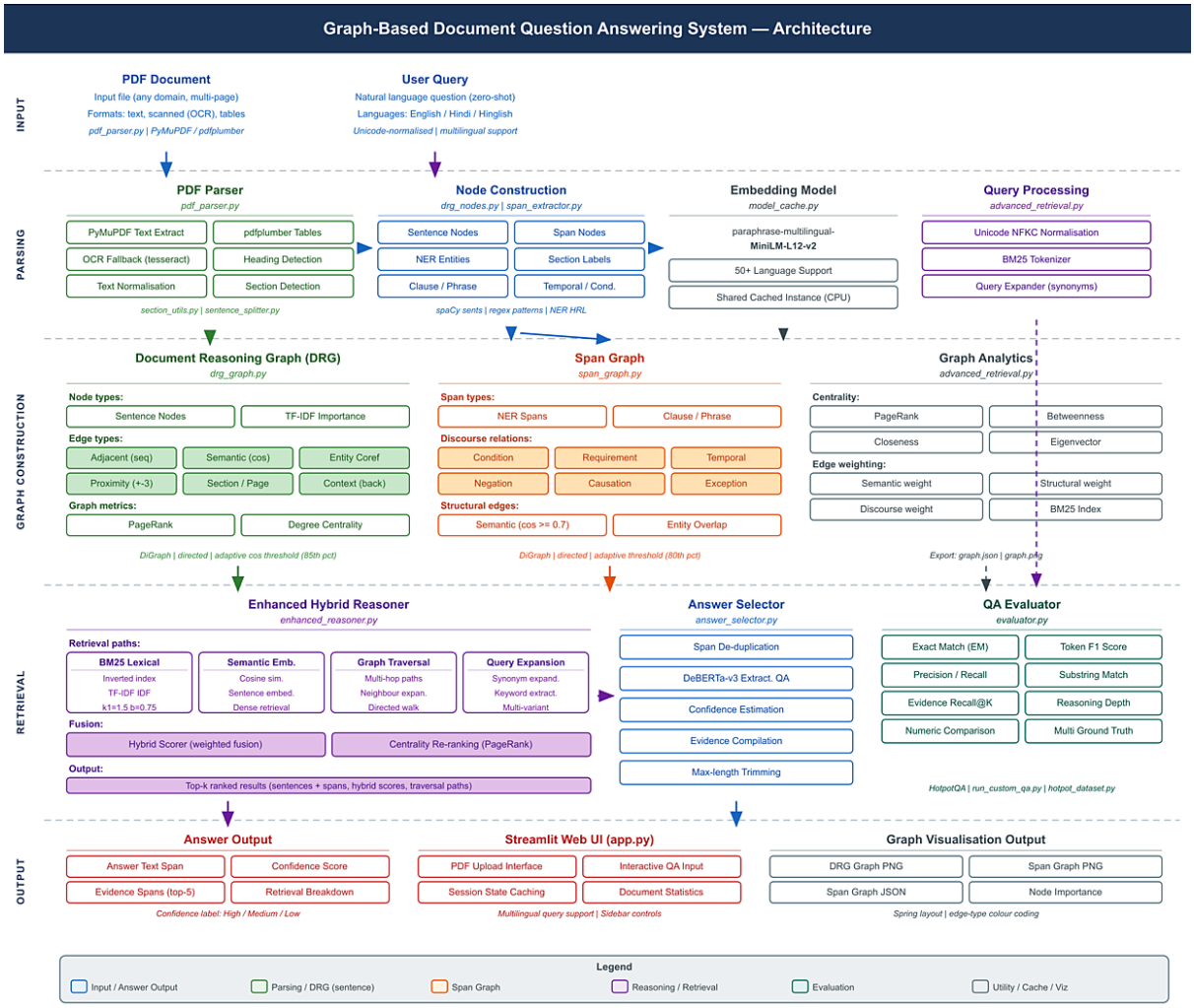

Figure 1: High-level pipeline from ingestion to evaluation.

stages including document ingestion, graph construction, hybrid retrieval, graph-based reasoning, and answer extraction.

The pipeline is designed to externalise reasoning through explicit graph structures rather than relying solely on implicit reasoning within large language models. This design improves interpretability and allows reasoning paths to be inspected through supporting spans and graph traversal traces.

4.2 Document Ingestion and Parsing

The system begins by ingesting documents in multiple formats such as PDF, text, or web pages. A document parser extracts raw text while preserving structural information such as sections, paragraphs, and sentence boundaries.

If the document contains scanned content, an OCR module is used to convert images into machine-readable text. The parsed content is then normalised through standard preprocessing steps including whitespace normalisation, sentence segmentation, and tokenisation.

Each sentence becomes a candidate textual span that will later serve as a node within the reasoning graph.

4.3 Node Construction and Representation

After preprocessing, textual units are converted into nodes within the Document Reasoning Graph (DRG). Each node represents a sentence-level span and stores the following attributes:

- Raw text of the span
- Section identifier and positional metadata
- Sentence embedding generated using a SentenceTransformer model, (Reimers and Gurevych, 2019)

- Linguistic annotations such as named entities and key phrases

These embeddings provide a semantic representation that allows similarity comparisons between spans and user queries.

4.4 Graph Construction

The Document Reasoning Graph $G = (V, E)$ is constructed by connecting span nodes through multiple types of edges capturing structural and semantic relationships.

Edge types include:

- **Adjacency edges** connecting neighboring sentences
- **Section edges** linking spans within the same section
- **Semantic similarity edges** based on cosine similarity between sentence embeddings, added when the similarity exceeds a threshold of 0.75
- **Discourse edges** capturing logical relationships such as conditions or explanations, identified using a keyword-based heuristic that flags sentence pairs containing discourse markers (e.g., *however*, *therefore*, *because*)
- **Entity edges** linking spans that reference the same entity

Graph analytics such as PageRank (Page et al., 1999) and centrality scores are then computed to estimate the relative importance of nodes within the document structure.

4.5 Span Graph Construction

In addition to the Document Reasoning Graph (DRG), the system constructs a secondary graph structure called the *Span Graph*. While the DRG operates at the sentence level, the Span Graph captures finer-grained semantic units extracted from sentences, enabling more precise reasoning over entities, phrases, and numeric expressions.

From each sentence node in the DRG, candidate spans are extracted using linguistic and statistical heuristics. These spans typically include named entities, noun phrases, and important factual expressions such as dates, quantities, or titles. Each extracted span becomes a node in the Span Graph and inherits contextual metadata from its parent sentence, including section identifiers and positional offsets.

Edges between span nodes are created based on several relationships:

- **Entity co-reference edges**, linking spans that refer to the same entity.
- **Semantic similarity edges**, computed using cosine similarity between span embeddings.
- **Contextual edges**, connecting spans that originate from the same sentence or neighboring sentences.
- **Discourse edges**, capturing logical relations such as cause, condition, or temporal ordering.

The Span Graph therefore acts as a fine-grained reasoning layer that complements the DRG. During retrieval and graph traversal, candidate evidence nodes can originate from either sentence-level DRG nodes or span-level nodes. This allows the system to perform reasoning at multiple levels of granularity.

Integrating both graphs improves the system's ability to identify precise answer candidates, especially for questions that require entity-level reasoning or extraction of short factual spans such as names, dates, and quantities.

4.6 Hybrid Retrieval Module

Given a user query, the system performs candidate retrieval using a hybrid scoring mechanism that combines multiple signals:

- Lexical similarity using BM25
- Semantic similarity using dense embeddings
- Graph centrality scores derived from the DRG and Span Graph

The hybrid retrieval score integrates these signals through weighted normalization, enabling the system to capture both keyword-level and semantic relevance. This stage produces a ranked list of candidate spans that are likely to contain relevant evidence.

4.7 Graph-Based Reasoning

To support multi-hop reasoning, the system performs traversal over the reasoning graph starting from the retrieved seed nodes. A bounded depth traversal explores neighboring nodes while prioritising edges associated with discourse relations or shared entities.

During traversal, candidate nodes are re-scored using a combination of semantic similarity, centrality measures, and token overlap with the query. This process allows the system to collect supporting evidence that may not appear in the initial retrieval results but is connected through graph relationships.

The resulting evidence subgraph represents the reasoning path used to derive the final answer.

4.8 Answer Extraction

Once candidate contexts are identified, the system applies an extractive question answering model to generate answer spans. A transformer-based QA model processes the query together with each candidate context and predicts the start and end positions of the answer within the text.

Multiple contexts are evaluated, and the system selects the answer with the highest confidence score while considering retrieval scores and graph-based evidence.

4.9 Answer Verification and Provenance

To improve reliability, the system attaches provenance information to each answer. Supporting spans are selected from the reasoning subgraph based on retrieval score thresholds and token overlap with the predicted answer.

This allows the system to provide:

- The predicted answer
- Supporting evidence spans
- Confidence score
- Reasoning path through the graph

Providing explicit reasoning traces improves interpretability and helps users verify the correctness of the system's predictions.

5 Methodology

5.1 Overview

We transform documents into a span/section graph, retrieve candidate spans with a hybrid lexical/semantic/centrality scorer, expand and traverse the graph to collect multi-hop evidence, apply cross-encoder re-ranking, and use an extractive QA pipeline to produce the final answer. Key model names and defaults used in the implementation are: sentence embedding model sentence-transformers/all-mpnet-base-v2, cross-encoder cross-encoder/mmarco-mMiniLMv2-L12-H384-v1 (optional), and extractive QA deepset/deberta-v3-large-squad2.

5.2 Preprocessing and Representation

Documents D are segmented into sections S and spans $v \in V$. Each span v stores text, metadata (section id, offsets, sentence id) and an embedding $\mathbf{e}_v \in \mathbb{R}^d$ computed by the SentenceTransformer. A graph $G = (V, E)$ is constructed; node attributes include text, embedding, and span metadata. Edge types include discourse, adjacency, section, citation, or co-reference. PageRank is computed with damping $\alpha_{PR} = 0.85$ using NetworkX.

5.3 Lexical Retrieval (BM25)

BM25 is used for lexical matching with parameters:

$$k_1 = 1.5, \quad b = 0.75$$

5.4 Centrality

We compute multiple centrality measures and combine them. For spans:

$$\mathrm{PR}(v) = \text{PageRank}_{\alpha=0.85}(v)$$
$$\mathrm{BC}(v) = \text{Betweenness}(v)$$

The combined centrality used by the system is

$$c(v) = \frac{\mathrm{PR}(v) + \mathrm{BC}(v)}{2}$$

5.5 Semantic Similarity

Semantic similarity is cosine similarity between query and span embeddings:

$$\mathrm{sim}_{sem}(q, v) = \cos(\mathbf{e}_q, \mathbf{e}_v)$$

5.6 Hybrid Retrieval Scoring

Signals (semantic similarity, BM25 lexical score, centrality) are each min–max normalised across candidate spans, then combined with fixed weights:

$$w_{sem} = 0.50, \quad w_{lex} = 0.30, \quad w_{cent} = 0.20$$

where $sem(v)$, $lex(v)$, $cent(v)$ denote the raw signals and $N[\cdot]$ denotes min–max normalisation, the retrieval score is:

$$\begin{aligned}\text{score}_{ret}(q,v) = w_{sem}N[\text{sim}_{sem}(q,v)] \\ +w_{lex}N[\text{BM25}(q,v)] \\ +w_{cent}N[c(v)]\end{aligned}$$

Practical retrieval bonuses used in code:

Lexical overlap adds a small bonus

$$\text{overlap_bonus} = \min(0.15, 0.03 \cdot \text{overlap_count})$$

and an additional $+0.03$ if the span's sentence also overlaps query tokens.

The retriever returns the top candidates by this combined score (by default top $2k$ before downstream reranking).

5.7 Sentence-Level Scoring

For sentence nodes the code uses an explicit weighted mixture:

$$\begin{aligned}\text{score}_{sent}(q,s) = 0.45 \cdot \text{sim}_{sem}(q,s) \\ +0.25 \cdot \text{lexical_overlap} \\ +0.20 \cdot \widehat{\text{BM25}}(q,s) \\ +0.10 \cdot c(s)\end{aligned}$$

(where $\widehat{\text{BM25}}$ denotes BM25 min–max normalised within the sentence set)

5.8 Graph-based Traversal and Multi-hop Evidence

Seed expansion starts from hybrid retrieval seeds (default 5).

Depth-limited traversal uses default $depth = 2$; for each node the algorithm visits at most the top 5 neighbors by centrality.

Node re-scoring during traversal uses the following:

$$\begin{aligned}\text{final}_{score}(v) = \text{sim}_{sem}(q,v) + 0.2 \cdot c(v) \\ +\text{discourse_bonus} \\ +\text{overlap_bonus}\end{aligned}$$

where

$$\text{overlap_bonus} = \min(0.2, 0.04 \cdot \text{token_overlap})$$

Traversal prioritizes discourse-type edges (e.g., condition, exception, temporal, same_sentence).

5.9 Cross-Encoder Re-ranking

When enabled, a cross-encoder (default cross-encoder/mmarco-mMiniLMv2-L12-H384-v1) scores (query, candidate span) pairs and re-ranks candidates by the cross-encoder score; top-K are returned for answer selection.

5.10 Answer Scoring, Selection and Calibration

Candidate assembly: Aggregating final candidates from multiple sources (hybrid retrieval, traversal, expansion, KG spans, etc.) and computing an internal span score (retriever/overlap/penalties) used to rank candidates.

Overlap and penalties: sentence spans get $+0.08$, overlap contributes $0.2 \times overlap_ratio$; long spans receive small negative length penalties (-0.0003 per token up to clipping).

Extractive QA pipeline: the top contexts are submitted to the QA pipeline (deepset/deberta-v3-large-squad2) to extract answer spans and QA confidence scores. The selection logic uses adaptive override margins:

$$\begin{aligned}OVERRIDE_MARGIN_HIGH = 0.20 \\ (\text{when primary_qa_score} \geq 0.45) \\ OVERRIDE_MARGIN_LOW = 0.05 \\ (\text{otherwise})\end{aligned}$$

A secondary context may override the primary if its QA confidence exceeds $max(best_qa_score, primary_qa_score + margin)$. This implements an implicit combined scoring that favors the retriever's primary passage unless a clearly better QA confidence is observed.

Multi-pass QA calibration:

Pass 1: individual contexts (primary protected as above).

Pass 2: if best_qa_score < 0.35, run QA on concatenated top-5 contexts to allow cross-context extraction.

Pass 3: if $best_qa_score < 0.50$, run a BM25-over-all-sentences search and QA on BM25-found sentences (fallback).

Final QA acceptance threshold: accept extracted QA answer if QA confidence ≥ 0.05 (otherwise fall back to passage-level answer).

Evidence and provenance: evidence spans are selected from the top-ranked retrieval passages using the rule $score \geq 0.6 \times best_retrieval_score$ or $overlap \geq 1$; at most 3 evidence passages are returned.

6 Evaluation Strategy

6.1 Evaluation Metrics

The performance of the proposed system is evaluated using several metrics commonly used in question answering and information retrieval tasks.

- **Exact Match (EM)**: Exact Match measures whether the predicted answer exactly matches the ground-truth answer. It is defined as:

$$EM = \begin{cases} 1 & \text{if predicted answer} = \text{ground truth} \\ 0 & \text{otherwise} \end{cases}$$

  The final EM score is the average over all questions.

- **F1-score**: The F1-score evaluates the overlap between the predicted answer and the ground-truth answer by combining precision and recall:

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

  where

$$Precision = \frac{|A_p \cap A_g|}{|A_p|},$$
$$Recall = \frac{|A_p \cap A_g|}{|A_g|}$$

  Here A_p represents the predicted answer tokens and A_g represents the ground-truth answer tokens.

- **Recall@k**: Recall@k measures whether the correct answer span appears within the top-k retrieved candidates. It evaluates the effectiveness of the retrieval stage before final answer selection.

$$Recall@k = \frac{N_{correct@k}}{N_{total}}$$

  where $N_{correct@k}$ denotes the number of queries for which the correct answer appears within the top-k retrieved candidates, and N_{total} denotes the total number of evaluated queries.

- **Graph Reasoning Depth**: This metric measures the number of reasoning hops required to reach the final answer within the reasoning graph. It provides insight into the system's ability to perform multi-hop reasoning across connected spans.

Difficulty	Exact Match (EM)	F1 Score
Easy	43.9%	56.3%
Medium	35.1%	42.9%
Hard	24.7%	35.0%

Table 1: Overall QA performance across difficulty levels.

- **Confidence**: The system assigns a confidence score to each predicted answer based on the softmax probability of the final answer score:

$$P(a|q) = \frac{\exp(\text{Score}(a|q))}{\sum_{a'} \exp(\text{Score}(a'|q))}$$

  Higher confidence values indicate stronger agreement between the language model score and graph-based evidence.

7 Results

We evaluate the proposed system on the stratified subset of the HotpotQA dataset containing 3,000 questions (1,000 each from the easy, medium, and hard splits). Performance is measured using Exact Match (EM) and token-level F1 score.

Table 1 presents the performance of the system across different difficulty levels.

7.1 Overall Performance

As expected, system performance decreases as question difficulty increases. Easy questions often require fewer reasoning steps and involve more direct lexical matches, while hard questions typically require deeper multi-hop reasoning across multiple spans.

7.2 Bridge vs Comparison Questions

HotpotQA contains two major reasoning types: *bridge* questions that require connecting information across documents, and *comparison* questions that require comparing properties of two entities.

Metric	Easy	Medium	Hard
EM Bridge	54.2%	40.6%	26.8%
EM Comparison	33.6%	29.6%	22.6%
F1 Bridge	64.5%	48.3%	37.8%
F1 Comparison	48.2%	37.4%	32.3%

Table 2: Performance comparison between bridge and comparison question types across difficulty levels.

Table 2 shows that the system performs consistently better on bridge questions than comparison

questions. Bridge questions often require locating supporting evidence through entity connections, which benefits from the graph traversal mechanism implemented in the Document Reasoning Graph and Span Graph. In contrast, comparison questions require reasoning over numerical or attribute-based relationships, which are more challenging for extractive QA models.

7.3 Analysis

Several trends can be observed from the results:

- Performance decreases steadily from easy to hard questions, highlighting the increasing difficulty of multi-hop reasoning across multiple documents.
- Bridge questions achieve higher EM and F1 scores across all difficulty levels. This indicates that graph-based traversal is effective in identifying chains of evidence that link related entities.
- Comparison questions show lower performance, suggesting that additional reasoning mechanisms may be required to explicitly model comparative relationships between entities.
- Despite the absence of task-specific training, the hybrid retrieval and graph reasoning framework is able to answer all questions in the evaluation subset and achieve F1 scores competitive for a zero-shot pipeline; for reference, fine-tuned models on HotpotQA typically report F1 scores in the range of 55–70%, and the system's easy-level F1 of 56.3% reflects that range without any task-specific training.

Overall, these results demonstrate that combining lexical retrieval, semantic similarity, and graph-based reasoning can effectively support multi-hop question answering over long documents.

8 Progress So Far

8.1 Completed Objectives

- Implemented the document ingestion and preprocessing pipeline capable of parsing PDF documents and extracting structured text.
- Constructed the *Document Reasoning Graph (DRG)* where sentences are represented as nodes and edges capture structural and semantic relationships within the document.
- Implemented span extraction methods to identify entities, noun phrases, and important factual expressions from sentences.
- Developed the *Span Graph* structure that represents fine-grained textual spans and their relationships.
- Implemented a hybrid retrieval mechanism combining BM25 lexical retrieval, dense semantic similarity using sentence embeddings, and graph centrality measures.
- Implemented graph traversal mechanisms for multi-hop evidence collection from both DRG and Span Graph structures.
- Integrated a transformer-based extractive QA model to generate final answer spans from retrieved contexts.
- Established an evaluation pipeline using the HotpotQA dataset and implemented evaluation metrics including EM, F1, and Recall@k.

8.2 Ongoing Work

- Enriching the Span Graph with **inter-entity relationships**. This involves identifying entity mentions within spans and connecting spans that reference the same or related entities to improve reasoning across documents.
- Developing **question-type understanding mechanisms** that infer the expected answer type from the query. For example, questions beginning with "Who" typically expect a person entity, while "Where" and "When" questions correspond to location and temporal entities respectively.
- Integrating these question-type signals into the answer selection stage so that candidate answers are filtered or prioritized based on the expected entity category.

8.3 Planned Work

- Integrating an external **knowledge graph**, (Lin and Zhang, 2021) into the reasoning pipeline to provide additional background knowledge and enable connections between document spans and structured world knowledge.

- Extending the system to support **multilingual question answering**, (Clark et al., 2020) specifically enabling queries in Hindi, Hinglish, and English.

- Improving entity linking and cross-document entity resolution to better connect evidence spans across different documents.

9 Tentative Timeline

The planned development schedule for the proposed system is illustrated in Figure 2. The project is organised into three major phases: initial system design and infrastructure development (Weeks 1–5), implementation and integration of the core reasoning modules (Week 6), and advanced feature development and evaluation (Weeks 7–10).

During the first phase (Weeks 1–5), the focus is on establishing the foundational components of the system. This includes conducting a literature survey and analysing existing benchmarks such as HotpotQA, followed by the development of the document processing pipeline. Key tasks include implementing PDF parsing, sentence segmentation, and section labelling. These steps are used to construct the Document Reasoning Graph (DRG), where sentences are represented as nodes and connected through structural and semantic edges. In parallel, span extraction methods are implemented to construct the Span Graph, which captures fine-grained textual units such as entities and key phrases.

The second phase (Week 6) focuses on integrating the hybrid retrieval and answer selection modules. In this stage, the system combines lexical retrieval using BM25 with dense semantic similarity and graph traversal mechanisms to identify relevant evidence spans. The extractive QA module is then integrated to generate answer predictions from retrieved contexts.

The final phase (Weeks 7–10) introduces advanced reasoning and multilingual capabilities. Planned work includes integrating external knowledge graphs to enrich the reasoning process and adding question-type understanding mechanisms to guide answer selection. Additional efforts will focus on enabling multilingual support for Hindi, Hinglish, and English queries. The final weeks are dedicated to implementing multi-hop evidence chain construction, conducting comprehensive experiments, performing ablation studies, and preparing the final evaluation report.

This staged development strategy ensures that the system evolves from core infrastructure and graph construction toward advanced reasoning capabilities and comprehensive evaluation.

Limitations

The proposed system demonstrates the effectiveness of hybrid retrieval and graph-based reasoning for document question answering. However, several limitations remain:

- **Fixed Retrieval Weights:** The hybrid retrieval mechanism uses manually defined weights for combining semantic similarity, lexical matching, and graph centrality signals. Since these parameters are not learned through training, the system may not adapt optimally across different datasets or domains.

- **Dependence on Preprocessing Quality:** The performance of the reasoning graphs depends heavily on accurate sentence segmentation, span extraction, and edge construction. Errors introduced during preprocessing may propagate through the graph and affect retrieval and reasoning.

- **Heuristic Graph Traversal:** The current graph traversal strategy relies on heuristic constraints such as fixed traversal depth and neighbor limits. While these constraints improve efficiency, they may prevent the discovery of longer reasoning chains.

- **Limited External Knowledge:** The span graph primarily captures relationships derived from the document itself. Questions requiring external background knowledge or implicit entity relationships may therefore be difficult to answer.

- **Single-Dataset Evaluation:** The system is evaluated only on the HotpotQA dataset without additional training data. As a result, the system's performance may vary when applied to other datasets or document domains.

Future work will focus on integrating external knowledge graphs, enriching the span graph with inter-entity relationships, and incorporating question-type understanding mechanisms to guide answer selection based on expected entity types.

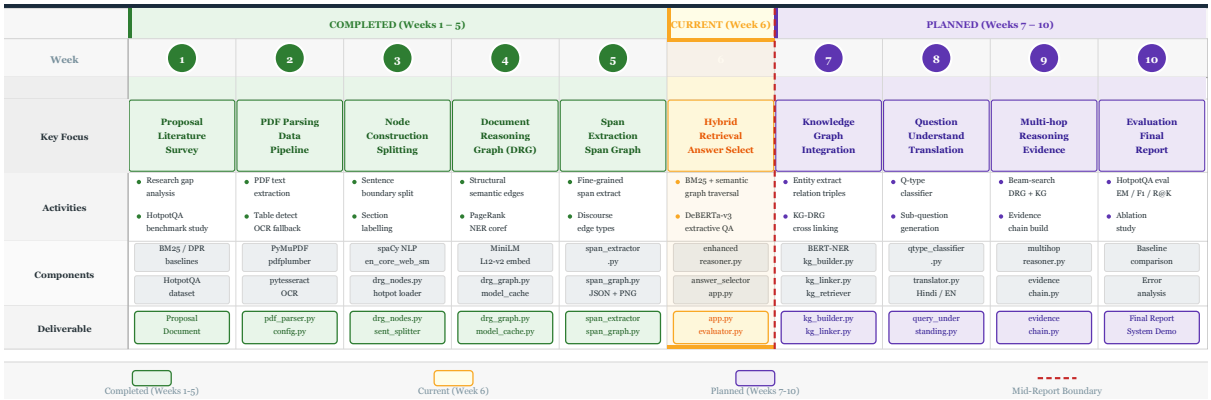

Figure 2: Tentative project timeline.

References

Jonathan H. Clark, Eunsol Choi, Michael Collins, Dan Garrette, Tom Kwiatkowski, Vitaly Nikolaev, and Jennimaria Palomaki. 2020. TyDi QA: A benchmark for information-seeking question answering in typologically diverse languages. *Transactions of the Association for Computational Linguistics*, 8:454–470.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. *Preprint*, arXiv:1810.04805.

Ming Ding, Chang Zhou, Qibin Chen, Hongxia Yang, and Jie Tang. 2019. Cognitive graph for multi-hop reading comprehension at scale. *Preprint*, arXiv:1905.05460.

Yuwei Fang, Siqi Sun, Zhe Gan, Rohit Pillai, Shuohang Wang, and Jingjing Liu. 2020. Hierarchical graph network for multi-hop question answering. *Preprint*, arXiv:1911.03631.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. Realm: Retrieval-augmented language model pre-training. *Preprint*, arXiv:2002.08909.

Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. 2020. Dense passage retrieval for open-domain question answering. *Preprint*, arXiv:2004.04906.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Preprint*, arXiv:2005.11401.

Xiaofei Lin and Yuzhi Zhang. 2021. Knowledge graph post-processing for qa systems. In *Proceedings of the 2021 7th International Conference on Computing and Artificial Intelligence*, ICCAI '21, page 332–338, New York, NY, USA. Association for Computing Machinery.

Yi Luan, Dave Wadden, Luheng He, Amy Shah, Mari Ostendorf, and Hannaneh Hajishirzi. 2019. A general framework for information extraction using dynamic span graphs. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 3036–3046, Minneapolis, Minnesota. Association for Computational Linguistics.

Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. The pagerank citation ranking : Bringing order to the web. In *The Web Conference*.

Anish Pahilajani, Devasha Trivedi, Jincen Shuai, Khin S. Yone, Samyak Rajesh Jain, Namyong Park, Ryan A. Rossi, Nesreen K. Ahmed, Franck Dernoncourt, and Yu Wang. 2024. Grs-qa – graph reasoning-structured question answering dataset. *Preprint*, arXiv:2411.00369.

Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. *Preprint*, arXiv:1908.10084.

Stephen E. Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: Bm25 and beyond. *Found. Trends Inf. Retr.*, 3:333–389.

Ming Tu, Guangtao Wang, Jing Huang, Yun Tang, Xiaodong He, and Bowen Zhou. 2019. Multi-hop reading comprehension across multiple documents by reasoning over heterogeneous graphs. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2704–2713, Florence, Italy. Association for Computational Linguistics.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium. Association for Computational Linguistics.